

Talents Connect

Application Next.js full-stack (App Router) — un seul projet pour le frontend, les server actions et l'accès aux données, connecté à une base PostgreSQL via Prisma.

Next.js 16 (App Router)

TypeScript

PostgreSQL

Prisma 7

NextAuth (Auth.js) v5

Tailwind CSS v4

Zod

01 Stack technique

COUCHE	CHOIX	RÔLE
Framework	Next.js 16, App Router	Rendu serveur (RSC), routage par dossier, server actions
Langage	TypeScript	Typage de bout en bout, y compris le client Prisma généré
Base de données	PostgreSQL	Stockage relationnel de l'ensemble des données
ORM	Prisma 7 (driver adapter <code>@prisma/adapters-pg</code>)	Schéma, migrations, requêtes typées
Authentification	NextAuth (Auth.js) v5, stratégie JWT	Comptes Credentials + OAuth (Google/LinkedIn)
Style	Tailwind CSS v4 (jetons CSS)	Design system de la marque Talents Consult
Validation	Zod	Validation des formulaires côté serveur
Mots de passe	bcryptjs	Hachage des mots de passe Credentials

02 Structure du projet

Les routes sont organisées par façade — chaque groupe de routes correspond à un espace applicatif protégé par rôle.

```
# extrait – arborescence simplifiée
prisma/
  schema.prisma      # 24 modèles, migrations versionnées
  seed.ts            # compte admin cabinet + offres de démo
src/
  proxy.ts           # middleware – protège /client, /cabinet, /candidate
  app/
    (public-jobs)/    # job board public : /, /results, /jobs/[id],
  /trainings...
    (auth)/           # /login, /register, /register/cabinet, /api/auth/*
    client/           # espace Client (protégé, rôle CLIENT)
    cabinet/         # espace Cabinet (protégé, rôle CABINET)
    candidate/       # espace Candidat (protégé, rôle CANDIDATE)
  components/
    ui/              # Button, Card, Tag, Field, Tabs – primitives
partagées
  mandats/ client/ cabinet/ candidate/ jobs/ training/ layout/
  lib/
    auth.ts          # config NextAuth complète (Node) – providers,
callbacks
  auth.config.ts     # config partagée, compatible Edge (utilisée par
proxy.ts)
  prisma.ts          # instance PrismaClient singleton
  rbac.ts            # requireRole() / requireAdmin() / getSessionUser()
  storage.ts         # upload de fichiers (CV) – adaptateur disque local
  notify.ts          # écriture Notification + point d'extension e-mail
  validations/       # schémas Zod par domaine
server/
  actions/           # logique métier ("use server") par domaine
  services/
    mandatStateMachine.ts # transitions de statut du mandat
```

03 Modèle de données

24 modèles Prisma, organisés en six domaines. Le détail des relations et un schéma visuel sont dans le document Architecture.

Identité & comptes

User , Company , ClientProfile , CabinetProfile , CabinetInvite , Candidate

Mandats

Mandat , MandatStatusEvent , MandatMessage , MandateApplication

Entretiens

Interview , InterviewSlot

Job board

JobPosting , JobApplication , DirectMessage , JobSponsorship , JobAlert

Formations

TrainingCourse , TrainingSession , Enrollment

Transverse

Notification

Le rôle utilisateur (Role : CLIENT / CABINET / CANDIDATE) porte à lui seul l'aiguillage vers la bonne façade — aucun rôle Admin séparé : les droits d'administration cabinet sont un simple booléen isAdmin sur CabinetProfile .

04 Authentification & autorisation

NextAuth est scindé en deux fichiers pour respecter la contrainte du middleware Next.js, qui s'exécute en *Edge Runtime* (aucun accès à Prisma/Node) :

- `lib/auth.config.ts` — callbacks `jwt / session` uniquement, aucun accès base de données. Utilisé directement par `proxy.ts`.
- `lib/auth.ts` — étend la config partagée avec les providers réels (Credentials, Google, LinkedIn conditionnels) et un callback `signIn` qui interroge Prisma pour créer/retrouver l'utilisateur.

Protection des routes

`proxy.ts` (middleware) redirige vers `/login` tout accès non authentifié à `/client/*`, `/cabinet/*` ou `/candidate/*`. Chaque layout de façade revérifie ensuite le rôle côté serveur via `requireRole()`, et chaque server action revérifie indépendamment rôle et propriété des données — la protection ne repose jamais sur un seul point de contrôle.

Connexion sociale

Google et LinkedIn sont ajoutés dynamiquement aux providers uniquement si leurs variables d'environnement sont définies (`googleEnabled / linkedinEnabled`, exportées pour l'UI). Réservée au rôle Candidat : le callback `signIn` crée le `User` + `Candidate` au premier passage, et refuse la connexion si l'e-mail correspond déjà à un compte non-candidat.

05 Server actions & sécurité

Toute la logique métier vit dans `src/server/actions/*.ts`, sous forme de fonctions `"use server"` appelées directement depuis les composants client — pas de couche API REST intermédiaire. Chaque action suit le même schéma :

1. `requireRole("CLIENT")` // rejette si non connecté / mauvais rôle
2. Validation Zod des données reçues
3. Vérification de propriété // ex: `mandat.companyId === user.companyId`
4. Écriture Prisma (transaction si plusieurs tables)
5. `revalidatePath()` des pages concernées

Les transitions de statut de mandat passent systématiquement par `mandatStateMachine.ts`, qui valide la transition demandée contre une table de transitions autorisées et journalise chaque changement dans `MandatStatusEvent` — aucun code applicatif n'écrit `Mandat.status` directement.

06 Variables d'environnement

VARIABLE	REQUISE	RÔLE
<code>DATABASE_URL</code>	Oui	Connexion PostgreSQL
<code>AUTH_SECRET</code>	Oui	Secret de signature des sessions NextAuth
<code>NEXTAUTH_URL</code>	Oui	URL publique de l'application
<code>GOOGLE_CLIENT_ID</code> / <code>_SECRET</code>	Non	Active la connexion Google si définies
<code>LINKEDIN_CLIENT_ID</code> / <code>_SECRET</code>	Non	Active la connexion LinkedIn si définies
<code>SEED_CABINET_ADMIN_EMAIL</code> / <code>_PASSWORD</code>	Non	Surcharge les identifiants du compte admin créé par <code>db:seed</code>

07 Scripts npm

COMMANDE	EFFET
<code>npm run dev</code>	Serveur de développement (Turbopack)
<code>npm run build</code>	Build de production
<code>npm run db:migrate</code>	<code>prisma migrate dev</code> — applique le schéma en local
<code>npm run db:seed</code>	Crée le compte admin cabinet + les offres de démonstration
<code>npm run db:studio</code>	Ouvre Prisma Studio (exploration de la base)
<code>npm run vercel-build</code>	<code>prisma migrate deploy && next build</code> — build de déploiement en une étape

08 Vérification & tests

Aucune suite de tests automatisés n'a été mise en place ; la vérification a été faite en continu, à chaque fonctionnalité :

- `npm run build` et `npx tsc --noEmit` systématiques avant chaque livraison.
- `npm run lint` (ESLint, config Next.js) systématique.
- Parcours de bout en bout dans un navigateur réel (Playwright headless) pour chaque fonctionnalité livrée : création de compte, dépôt de mandat, sourcing, entretiens, clôture, candidature, messagerie, sponsoring, alertes — avec vérification explicite des droits d'accès (un client ne peut pas voir le mandat d'un autre, etc.).

Recommandation

Ajouter une suite de tests automatisés (Vitest/Playwright) avant toute mise en production réelle — la couverture actuelle repose sur des vérifications manuelles répétées, pas sur des tests exécutables en continu.

09 Déploiement

1. Créer une base PostgreSQL managée (Neon ou Supabase, offre gratuite suffisante pour démarrer).
2. Importer le dépôt GitHub dans un nouveau projet Vercel.
3. Renseigner les variables d'environnement de la section 6 dans les réglages du projet Vercel.
4. Vercel détecte automatiquement `vercel-build` et l'exécute — migrations puis build, en une étape.

Détail pas-à-pas (y compris l'obtention des identifiants Google/LinkedIn) dans `README.md` à la racine du dépôt.

10 Dette technique connue

SUJET	ÉTAT	RECOMMANDATION
Tests automatisés	Aucun	Ajouter Vitest (unitaire) + Playwright (bout en bout)
E-mail transactionnel	Non implémenté (<code>lib/notify.ts</code> prêt à l'accueillir)	Brancher un fournisseur (Resend, SendGrid...)
Stockage de fichiers	Disque local (<code>lib/storage.ts</code>)	Basculer vers un stockage objet (S3-compatible) avant mise à l'échelle
Paiements	Suivi manuel (formations, sponsoring)	Intégrer une passerelle une fois un compte marchand disponible
Recherche	Filtrage SQL simple	Suffisant au volume actuel ; revoir si le catalogue grossit fortement

